LAPORAN PRAKTIKUM
INTERNET OF THINGS

Disusun oleh :

Nama : Restu anggara

NPM : 25782051

Kelas : TRI-3B

Dosen :

Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.

Agus Ambarwari, S.Pd.,M.Kom.

Andi Saptono,S.Kom

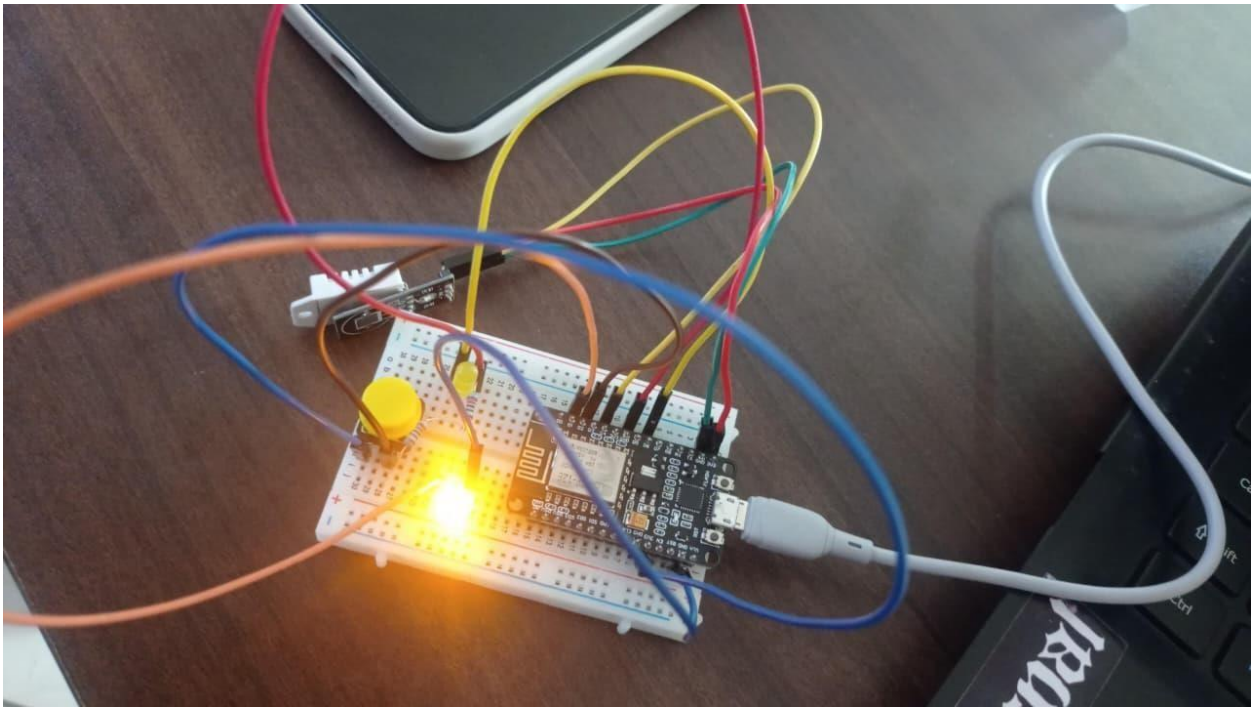
TEKNOLOGI REKAYASA INTERNET

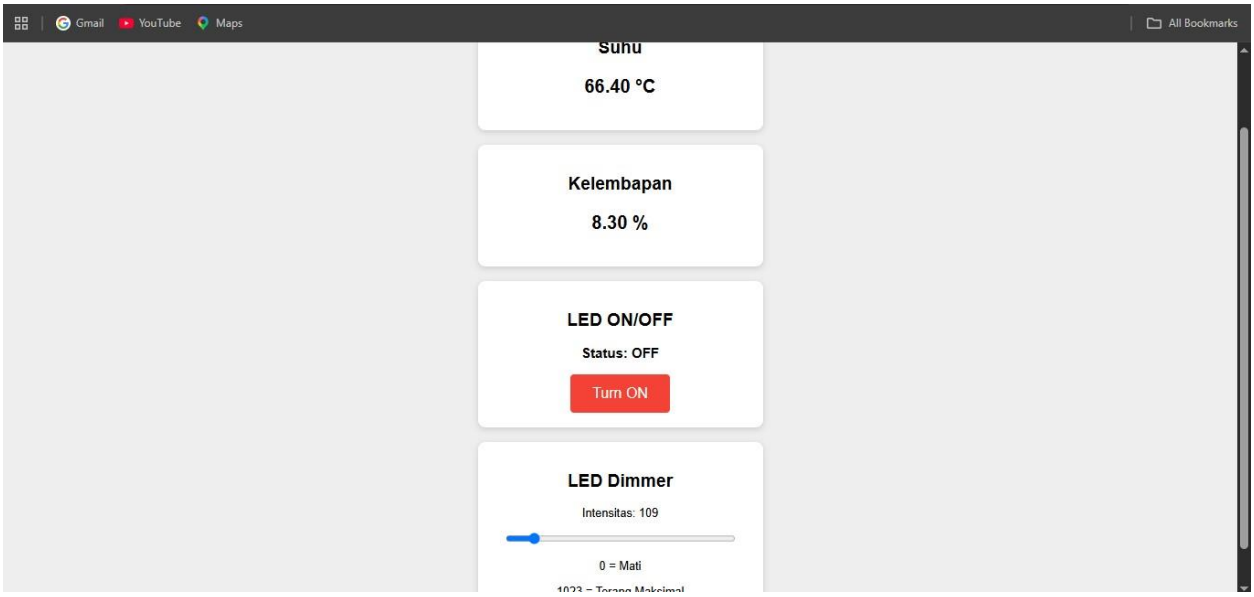
JURUSAN TEKNOLOGI INFORMASI POLITEKNIK

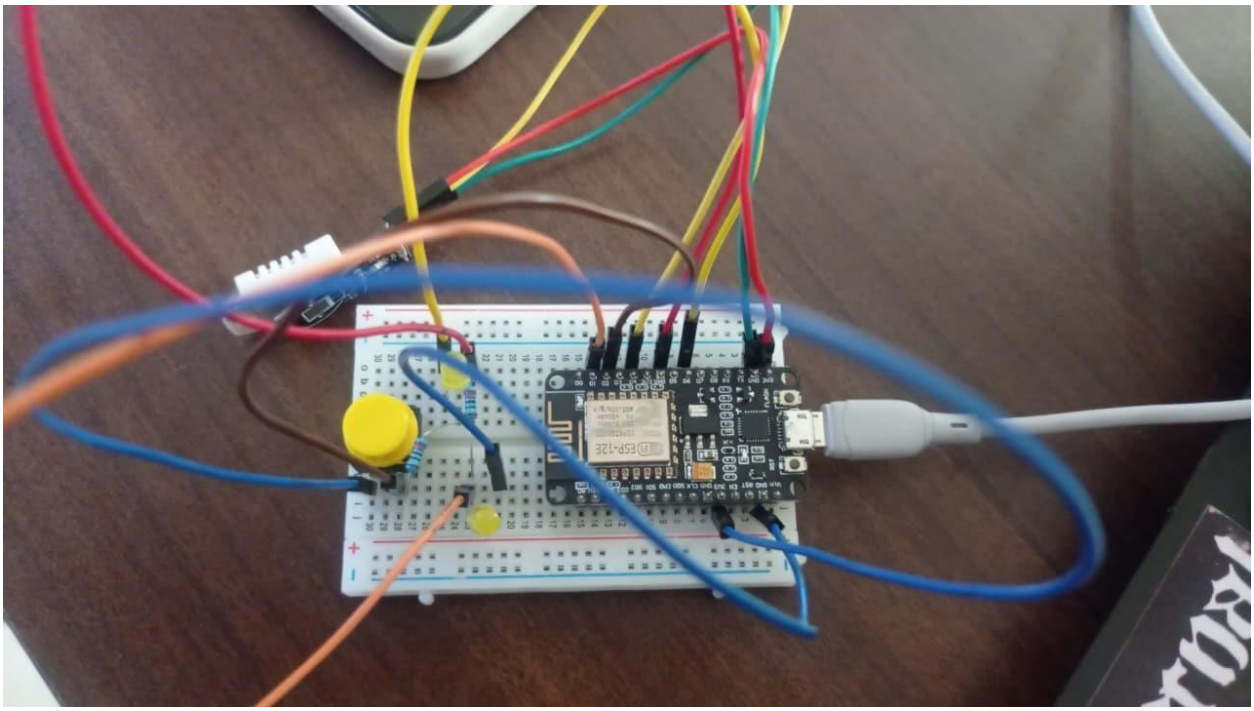
NEGERI LAMPUNG

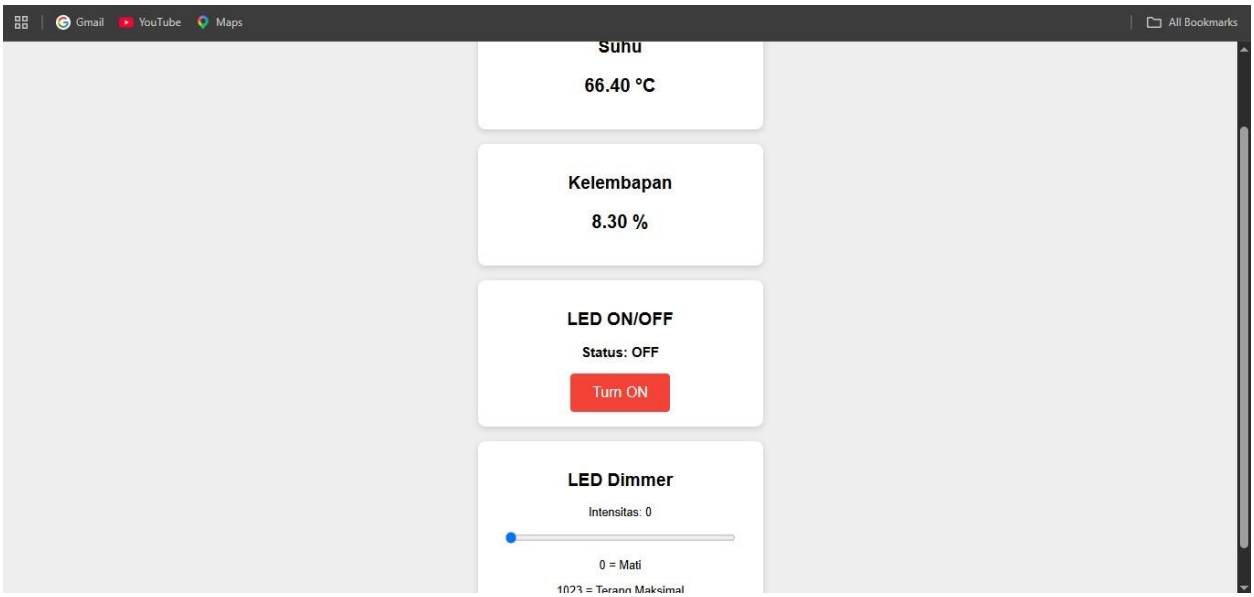
2026/2027

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*









Kode perintah:

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>
 const char* ssid = "cihuyyy";
const char* password =
"12345678";

// ================= PIN =================
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)
const byte pinLED = 5;        // D1 (GPIO 5) - LED PWM

// ================= DHT =================
#define DHTTYPE DHT22
DHT dht(dhtPin, DHTTYPE);

// ================= VARIABEL ================= bool
ledState = false;

String currentTemp = "--";
String currentHum = "--";
 int pwmValue =
0;
 int buttonState = LOW;
int lastButtonState =
LOW;
 unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay =
50;
 unsigned long lastTime =
0;

// ================= WEB SERVER =================
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ================= HTML =================
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Smart Lighting IoT</title>

  <style>     body {
font-family: Arial;
text-align: center;
background: #eeeeee;
    }

    .card {       background: white;
margin: 20px auto;       padding: 20px;
max-width: 350px;       border-radius: 10px;
box-shadow: 0 2px 8px rgba(0,0,0,0.15);
    }      button {
padding: 15px 30px;
font-size: 20px;
border-radius: 5px;
cursor: pointer;
color: white;
border: none;
    }

    .btn-on {       background-
color: #4CAF50;
    }

    .btn-off {
      background-color: #f44336;
    }
    input[type=range] {
width: 90%;
    }

    .value {
font-size: 25px;
font-weight: bold;
    }
```

```
  </style>
</head>

<body>

  <h1>Smart Room</h1>

  <!-- ================ SUHU ================ -->
  <div class="card">
    <h2>Suhu</h2>
    <p class="value">
      <span id="tempValue">--</span> &deg;C
    </p>
  </div>

  <!-- ================ KELEMBAPAN ================ -->
  <div class="card">
    <h2>Kelembapan</h2>
    <p class="value">
      <span id="humValue">--</span> %
    </p>
  </div>

  <!-- ================ LED ON OFF ================ -->
  <div class="card">
    <h2>LED ON/OFF</h2>

    <h3>
      Status:
      <span id="ledStatus">OFF</span>
    </h3>

    <button
id="toggleBtn"
class="btn-off"
onclick="toggleLed()">
      Turn ON
    </button>
  </div>

  <!-- ================ LED DIMMER ================ -->
  <div class="card">
    <h2>LED Dimmer</h2>
```

```
      Intensitas:
      <span id="pwmValue">0</span>
    </p>

    <input       type="range"
min="0"       max="1023"
value="0"       id="pwmSlider"
oninput="sendPWM(this.value)"
    >

    <p>0 = Mati</p>
    <p>1023 = Terang Maksimal</p>
  </div>

  <script>

    // ================= WEBSOCKET =================
     var gateway =
`ws://${window.location.hostname}/ws`;
     var
websocket;
     window.addEventListener('load',
onLoad);
     function onLoad(event)
{       initWebSocket();
    }
    function initWebSocket() {

      websocket = new WebSocket(gateway);
       websocket.onopen =
onOpen;

      websocket.onclose = onClose;
       websocket.onmessage =
onMessage;
    }      function
onOpen(event) {
       console.log("WebSocket
Terkoneksi");
```

```
    }      function
onClose(event) {
       console.log("WebSocket
Terputus");
       setTimeout(initWebSocket,
2000);
    }


    // ================= LED ON OFF =================
     function toggleLed()
{

websocket.send("toggle");
    }


    // ================= LED PWM =================
     function sendPWM(value)
{
       document.getElementById("pwmValue").innerHTML       =
value;
       websocket.send("pwm," +
value);
    }


    // ================= TERIMA DATA =================
     function onMessage(event)
{
       var dataObj =
JSON.parse(event.data);

      // ---------- SUHU ----------

      if (dataObj.suhu !== undefined) {
         document.getElementById("tempValue").innerHTML
=           dataObj.suhu;
      }


      // ---------- KELEMBAPAN ----------
```

```
      if (dataObj.hum !== undefined) {
         document.getElementById("humValue").innerHTML
=           dataObj.hum;
      }


      // ---------- LED ON OFF ----------
       if (dataObj.led !== undefined)
{
         var btn =
document.getElementById("toggleBtn");
         var status =
document.getElementById("ledStatus");


        if (dataObj.led == "1") {
           status.innerHTML =
"ON";
           btn.innerHTML = "Turn
OFF";
           btn.className = "btn-
on";

        } else {
           status.innerHTML =
"OFF";
           btn.innerHTML = "Turn
ON";

          btn.className = "btn-off";
        }
      }


      // ---------- PWM ----------

      if (dataObj.pwm !== undefined) {
         document.getElementById("pwmValue").innerHTML
=           dataObj.pwm;
         document.getElementById("pwmSlider").value
=
```

```
          dataObj.pwm;
      }
    }

  </script>

</body>
</html>
)rawliteral";


// ========================================================
// NOTIFY CLIENT
// Mengirim data LED, suhu, kelembapan dan PWM
// ========================================================
 void notifyClients()
{

  String jsonString =
    "{\"led\":\"" +
    String(ledState ? 1 : 0) +
    "\", ";
   jsonString +=
"\"suhu\":\"" +
currentTemp +
    "\", ";
   jsonString +=
"\"hum\":\"" +
currentHum +
    "\", ";
   jsonString
+=
    "\"pwm\":\"" +
    String(pwmValue) +
    "\"}";

  ws.textAll(jsonString);
}


// ========================================================
// HANDLE WEBSOCKET MESSAGE
// ========================================================
```

```
 void
handleWebSocketMessage(
void *arg,   uint8_t *data,
size_t len
) {

  AwsFrameInfo *info =
    (AwsFrameInfo*)arg;


  if (     info->final &&
info->index == 0 &&
info->len == len &&
info->opcode == WS_TEXT
  ) {
data[len] = 0;

    // Ubah data menjadi String
    String message =
      String((char*)data);


    // ================================================
// TOMBOL ON / OFF
    // ================================================
     if (message == "toggle")
{
       ledState =
!ledState;

notifyClients();
    }


    // ================================================
    // PWM
    // Format: pwm,512
    // ================================================
     else if (message.startsWith("pwm,"))
{

      String nilai =
```

```
        message.substring(4);


      pwmValue =
nilai.toInt();

      // Membatasi nilai PWM
      // 0 sampai 1023
       pwmValue =
constrain(
pwmValue,
          0,
          1023
        );


      // Mengatur brightness LED

analogWrite(
pinLED,
pwmValue
      );


      // Kirim status ke browser

notifyClients();
    }
  }
}


// ================================================== //
WEBSOCKET EVENT
// ==================================================

void onEvent(
  AsyncWebSocket *server,
  AsyncWebSocketClient
*client,   AwsEventType type,
void *arg,   uint8_t *data,
size_t len
) {
```

```
   switch (type)
{
     case
WS_EVT_CONNECT:

      Serial.printf(
        "Client WebSocket #%u terhubung\n",
client->id()
      );

      // Kirim status saat browser dibuka

notifyClients();

break;


    case WS_EVT_DISCONNECT:

      Serial.printf(
        "Client WebSocket #%u terputus\n",
client->id()
      );

break;


    case WS_EVT_DATA:

handleWebSocketMessage(
arg,         data,
len
      );

break;
  }
}


// ================================================== //
SETUP
// ==================================================
 void setup()
{
```

```
  Serial.begin(115200);


  // ================= BUTTON =================
   pinMode(
buttonPin,
    INPUT
  );


  // ================= LED ON/OFF =================

pinMode(
ledPin,
    OUTPUT
  );
digitalWrite(
ledPin,
    LOW
  );


  // ================= LED PWM =================

pinMode(
pinLED,
    OUTPUT
  );
analogWrite(
pinLED,
    0
  );


  // ================= DHT =================

  dht.begin();

  // ================= WIFI =================

  WiFi.mode(WIFI_STA);
```

```
  WiFi.begin(
ssid,
password
  );


  Serial.print(
    "Menghubungkan WiFi"
  );

  while (
    WiFi.status() != WL_CONNECTED
  ) {
delay(500);

    Serial.print(".");
  }


  Serial.println();

  Serial.print(
    "IP Address: "
  );

  Serial.println(
    WiFi.localIP()
  );


  // ================ WEB ================

server.on(
"/",
    HTTP_GET,
    [](AsyncWebServerRequest *request) {

      request->send_P(
200,
        "text/html",
index_html
      );
    }
```

```
  );


  // ================= WEBSOCKET =================

ws.onEvent(onEvent);

server.addHandler(&ws);

  // ================= START SERVER =================

server.begin();
  Serial.println(
    "Web Server Started"
  );
}


// ========================================================= //
LOOP
// =========================================================
 void loop()
{

ws.cleanupClients();

  // =========================================================
// LED ON / OFF
  // =========================================================

  digitalWrite(
ledPin,     ledState ?
HIGH : LOW
  );


  // =========================================================
// BUTTON FISIK
  // =========================================================
   int reading =
digitalRead(buttonPin);
```

```
  if (     reading !=
lastButtonState
  ) {
lastDebounceTime =
millis();
  }

  if (
    (millis() - lastDebounceTime)
    > debounceDelay
  ) {      if (
reading != buttonState
    ) {
buttonState =
reading;

      // Tombol ditekan = HIGH
       if (
buttonState == HIGH
      ) {
         ledState
=
          !ledState;


        // Update browser

notifyClients();
      }
    }
  }

  lastButtonState
=     reading;

  // ====================================================
  // DHT SETIAP 3 DETIK
```

```
  // ======================================================
   if
(
    (millis() - lastTime)
    > 3000
  ) {

    // Baca suhu
     float t =
dht.readTemperature();

    // Baca kelembapan
     float h =
dht.readHumidity();

    // Jika pembacaan valid

    if (
      !isnan(t) &&
      !isnan(h)
    ) {
      currentTemp =
        String(t);

      currentHum =
String(h);


      // Kirim data ke browser

      notifyClients();
    }

    lastTime =
millis();
  }
}
```

Penjelasan kode:

1. Menambahkan pin LED PWM

const byte pinLED = 5; // D1 (GPIO 5) int

pwmValue = 0;

Ditambahkan untuk menggunakan **LED kedua pada pin D1 (GPIO 5)**. Variabel pwmValue menyimpan nilai kecerahan LED dari **0–1023**.

2. Menambahkan tampilan slider pada Web

<input type="range"

min="0" max="1023"

value="0" id="pwmSlider"

oninput="sendPWM(this.value)">

Ditambahkan **slider** pada halaman web untuk mengatur tingkat kecerahan LED.

- min="0" → LED mati
- max="1023" → LED paling terang
- oninput → nilai dikirim secara langsung ketika slider digeser.

3. Menambahkan fungsi PWM pada JavaScript

function sendPWM(value) {

document.getElementById("pwmValue").innerHT

ML = value; websocket.send("pwm," + value);

}

Fungsi ini mengambil nilai dari slider kemudian mengirimkannya melalui **WebSocket**.

Contohnya jika slider berada di 512: pwm,512

Nilai tersebut dikirim ke ESP8266.

4. Menambahkan penerimaan perintah PWM

```
else if (message.startsWith("pwm,")) {
String nilai = message.substring(4);
pwmValue = nilai.toInt();   pwmValue =
constrain(pwmValue, 0, 1023);

  analogWrite(pinLED, pwmValue);
  notifyClients();
}
```

Bagian ini ditambahkan agar ESP8266 dapat menerima perintah pwm,nilai.

Kemudian nilai tersebut digunakan oleh:

analogWrite(pinLED, pwmValue); untuk mengatur

kecerahan LED menggunakan PWM.

5. Menambahkan status PWM ke JSON Pada notifyClients() ditambahkan: jsonString += "\"pwm\":\"" + String(pwmValue) + "\"}"; Tujuannya agar nilai PWM terbaru dikirim kembali ke browser.

Jadi angka pada slider tetap sesuai dengan kondisi LED.

6. Menambahkan konfigurasi LED PWM di setup() pinMode(pinLED, OUTPUT); analogWrite(pinLED, 0);

Digunakan untuk mengatur **D1 sebagai output** dan memastikan LED PWM awalnya dalam kondisi **mati**.

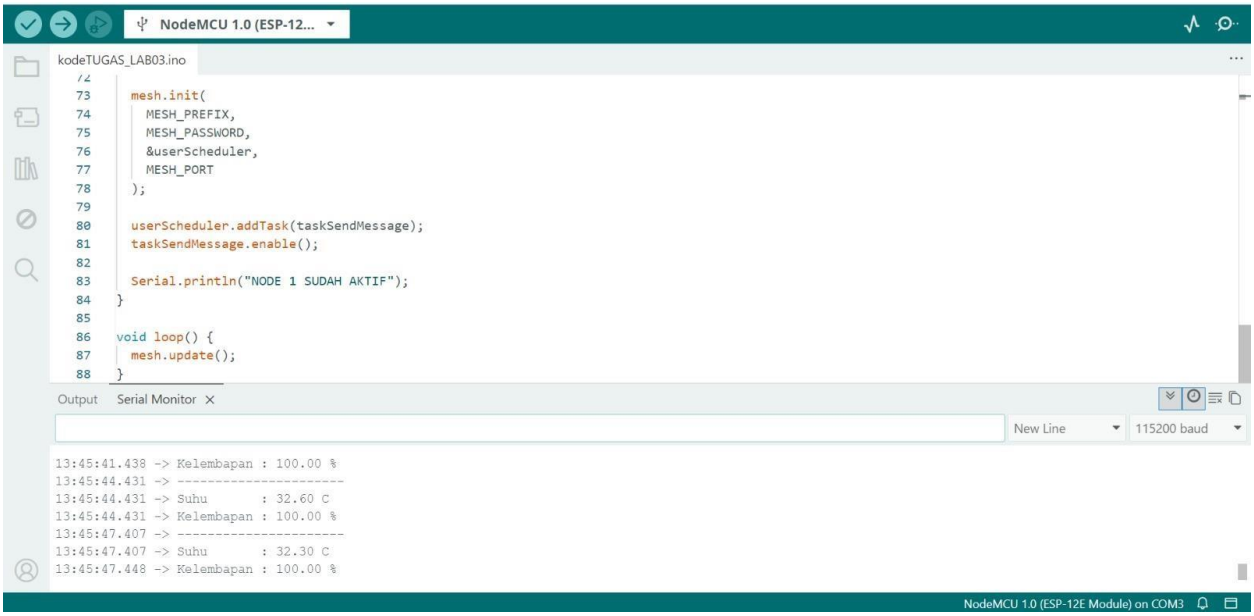
Kesimpulan:

Modifikasi program berhasil menambahkan fitur **Smart Lighting** dengan menggunakan LED pada pin D1 (GPIO 5) dan teknik **PWM**. Kecerahan LED dapat diatur melalui slider pada

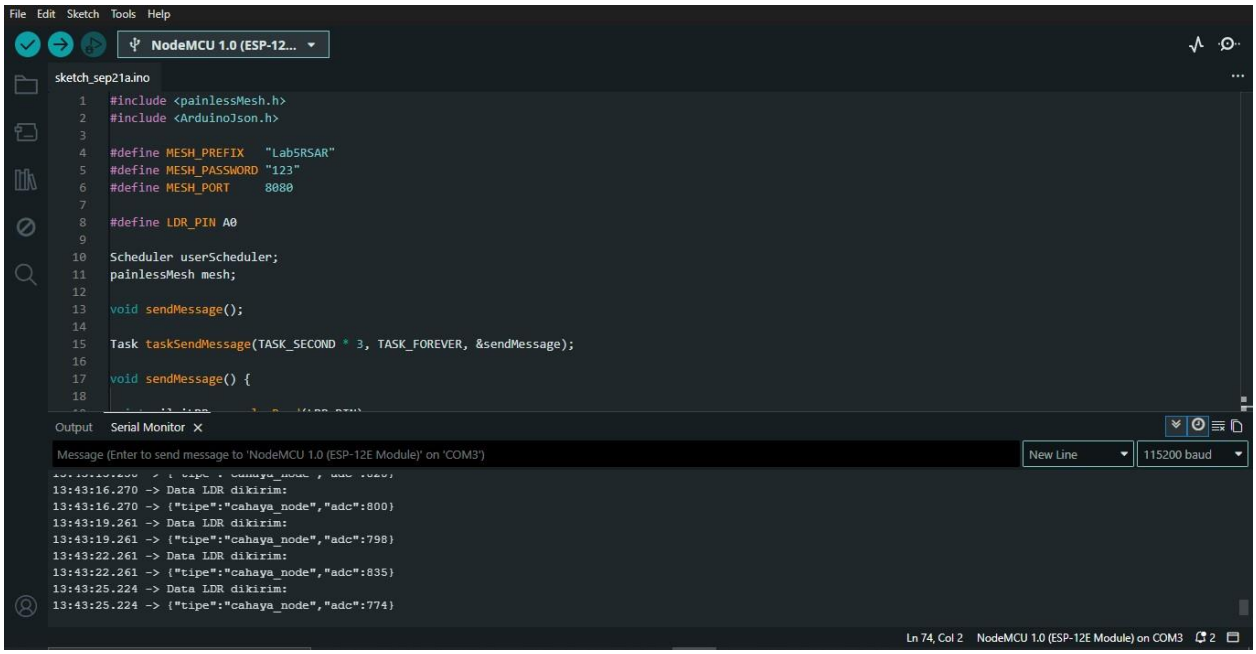
smartphone dengan nilai 0–1023 secara real-time menggunakan **WebSocket**. Fitur sebelumnya seperti kontrol LED ON/OFF, tombol fisik, serta pembacaan suhu dan kelembapan menggunakan DHT22 tetap dapat berjalan tanpa dihilangkan.

5.7 Tugas: Sistem Telemetri & Otomasi Terdistribusi Berbasis painlessMesh

Nodemcu 1-DHT22



Nodemcu2-LDR



Nodemcu3-LED

Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

Blink.ino

```
#include "painlessMesh.h"
#include <ArduinoJson.h>

#define MESH_PREFIX     "Lab5Medan"
#define MESH_PASSWORD   "Medan1234"
#define MESH_PORT       8080

painlessMesh mesh;

Scheduler userScheduler;

const int ledPin = D6;

// Menyimpan data terakhir dari Node 1 dan Node 2
float suhu = 0.0;
int adcCahaya = 1023;

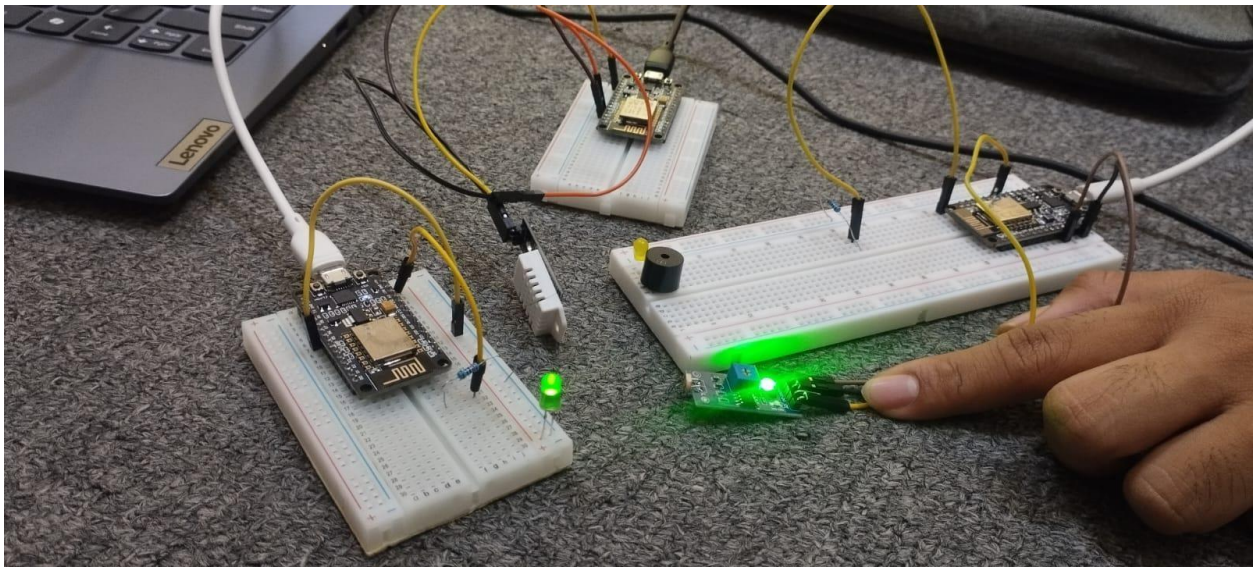
// Status apakah data sudah pernah diterima
bool suhuDiterima = false;
bool cahayaDiterima = false;
```

Output Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
Pesan JSON: {"tipe":"cahaya_node","adc":3}
Sumber: Node 2 - Sensor LDR
ADC Cahaya : 3

----- RULE ENGINE -----
Suhu > 31 C     : TIDAK
Cahaya < 300    : YA
KEPUTUSAN: LED ON
Alasan: Suhu tinggi ATAU kondisi gelap
========================================
```



Kesimpulan:

tiga NodeMCU dapat membentuk jaringan **mesh secara langsung tanpa menggunakan router Wi-Fi eksternal**. Node 1 mengirimkan data suhu dan kelembapan dari sensor DHT, sedangkan Node 2 mengirimkan data intensitas cahaya dari sensor LDR dalam format JSON.

Node 3 berperan sebagai **Hub Kontrol Aktuator** yang menerima dan memproses data dari node lain menggunakan *rule engine*. LED dapat menyala secara otomatis ketika suhu melebihi **31°C** atau kondisi cahaya gelap dengan nilai ADC di bawah **300**, dan mati ketika kondisi kembali normal. Selain itu, sistem dapat menunjukkan kemampuan **multi-hop** dan **self-healing**, sehingga jaringan tetap dapat berkomunikasi dan menyesuaikan koneksi ketika terjadi perubahan atau kegagalan pada salah satu node.